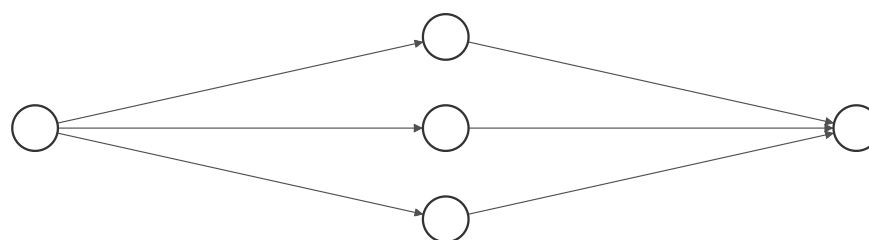


6.3 Neural Network with Julia Continued

```
In [44]: using Zygote
using Lux
using Random
using Distributions
using CairoMakie
using Optimisers
using Optimization
using OptimizationOptimisers
using ComponentArrays
```

So far, we have built relatively simple Neural Network with Julia. We can use the `Lux()` package in Julia to build very complex Neural Networks in Julia. I will give some examples here. To construct the Neural Network below,



Input Layer $\in \mathbb{R}^1$

Hidden Layer $\in \mathbb{R}^3$

Output Layer $\in \mathbb{R}^1$

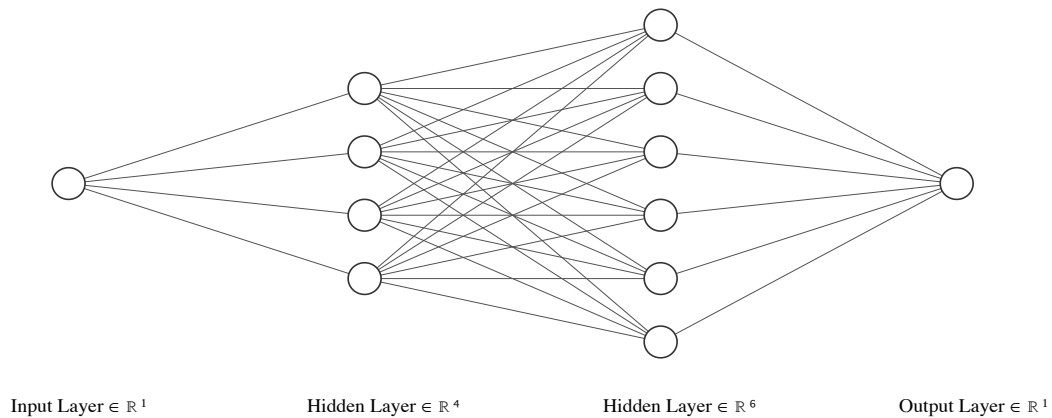
you can use the following Julia code

```
In [45]: model = Chain(Dense(1 => 2, tanh),
                      Dense(2=>1))
```

```
Chain(
  layer_1 = Dense(1 => 2, tanh),          # 4 parameters
  layer_2 = Dense(2 => 1),                # 3 parameters
)      # Total: 7 parameters,
      # plus 0 states.
```

This Neural Network has 1 hidden layer with 2 nodes, 1 input and 1 output.

The next Neural Network below also has 1 input and 1 output and has 2 hidden layers with 4 and 6 nodes respectively.



The code to construct this Neural Network using Lux in Julia is

```
In [46]: model = Chain(Dense(1 => 4, tanh),
                        Dense(4 => 6, tanh),
                        Dense(6 => 1))

Chain(
  layer_1 = Dense(1 => 4, tanh),           # 8 parameters
  layer_2 = Dense(4 => 6, tanh),           # 30 parameters
  layer_3 = Dense(6 => 1),                 # 7 parameters
)      # Total: 45 parameters,
      # plus 0 states.
```

So far, we have trained and conducted our predictions using all our data. This is a bit pointless because if we already have our data, then why do we need a model to do our predictions. The main reason for having a model is that you want to predict where you do not have data! Let's randomly pick 50% of our data set and then try to train our model to predict the other 50%. The data that we use to train our model is called the training dataset and the other data that we use to test our model is usually called the validation dataset.

Generate data using the function

$$h(x) = 2 \cos(\pi x) \exp(-x^2/5)$$

for $x \in [0, 5]$. We also add a bit of random noise to our data.

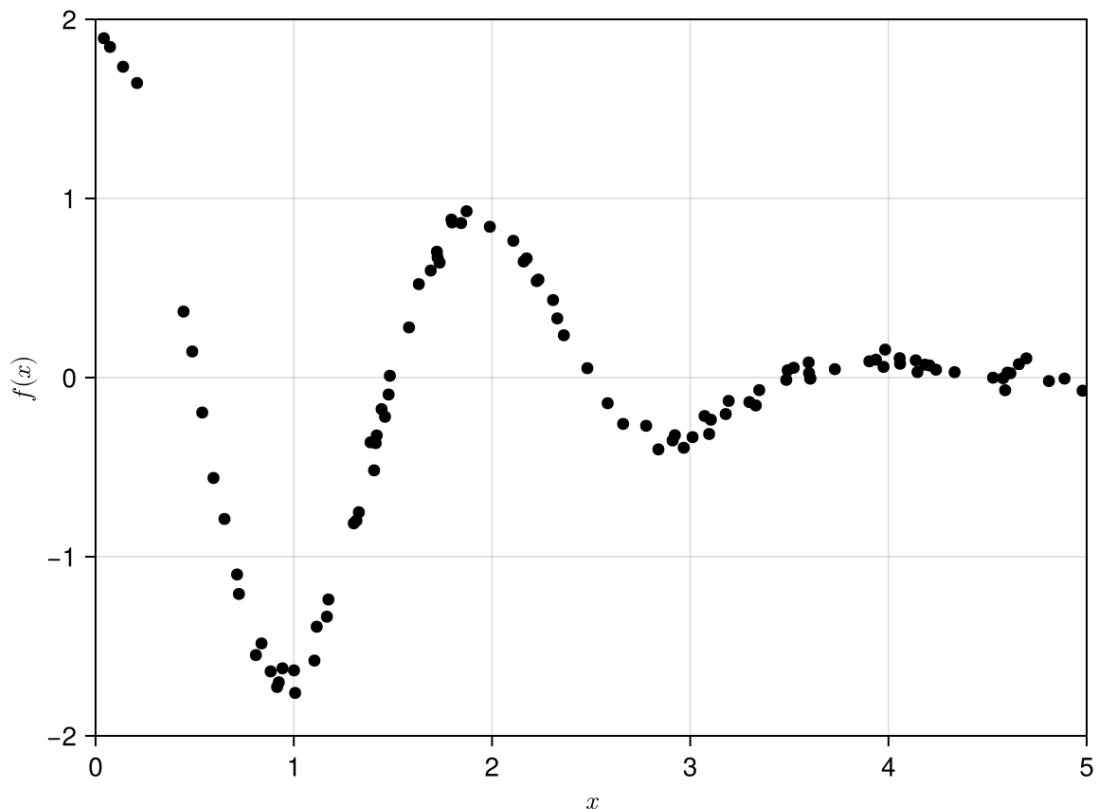
```
In [47]: N_SAMPLES=100
xdata=5*rand(N_SAMPLES)
h(x)=2.0*cos(pi *x )*exp(-x^2/5)
ydata=h.(xdata).+0.05*randn(N_SAMPLES)
```

100-element Vector{Float64}:

```
-0.17619228051694247
-1.7274756929111315
-0.7978215087224122
-0.19509082769413835
-1.7603478855316192
 0.8662223312958148
 0.5976289516342747
-0.8136904227827861
 0.04407056574901293
 0.36848817771805303
 ⋮
 0.3303569054410371
 0.9286929364896507
 0.05983233749487957
-0.25858989263775706
 0.8821441593661695
 0.763685119665301
 1.7352061081875674
 0.047096806322561045
-0.7892209947566635
```

Let's see how our data looks like

```
In [48]: fig = Figure(backgroundcolor = :transparent)
ax = CairoMakie.Axis(fig[1, 1], xlabel = L"x", ylabel = L"f(x)", background
scatter!(ax, xdata, ydata; color=:black, label="data")
#save("../figures/06p03NeuralNetworkWithJuliaPart2.png", fig)
fig
```



Let's select our training data randomly from our whole dataset . We will pick 50%.

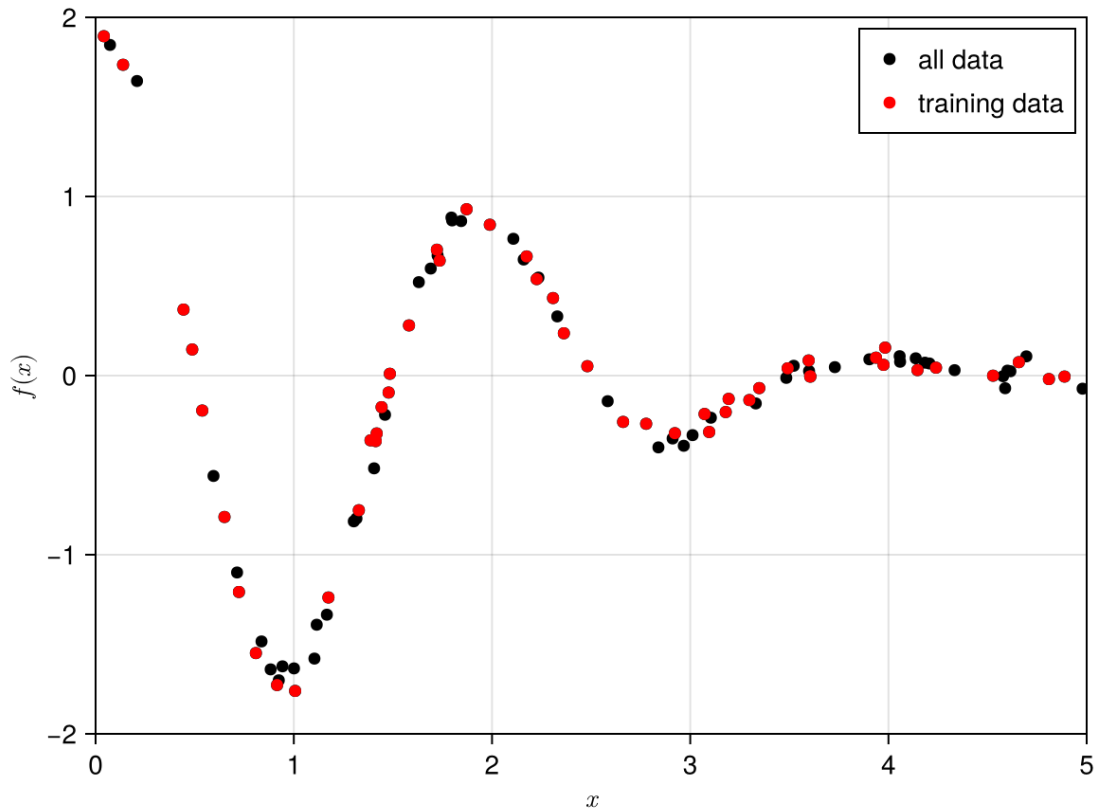
```
In [49]: #training_indices=rand(eachindex(xdata),round(Int,0.5*N_SAMPLES))
training_indices=shuffle(eachindex(xdata))[1:round(Int,0.5*N_SAMPLES)]
xtraining=xdata[training_indices]
ytraining=ydata[training_indices]
```

50-element Vector{Float64}:

```
 1.8944663310765575
-0.06977014543375529
 0.0101077746154251
-0.7892209947566635
-0.3613950081322883
 0.04407056574901293
-0.7517086975032635
-0.005763541687452252
-0.09458407956554007
 0.08435989368157731
 ⋮
-1.7274756929111315
 0.04058696766057787
-0.0005708323373534076
-0.019534510618887793
 0.9286929364896507
-0.25858989263775706
 1.7352061081875674
-1.7603478855316192
 0.05983233749487957
```

Plotting our training data with the original dataset

```
In [50]: fig = Figure(backgroundcolor = :transparent)
ax = CairoMakie.Axis(fig[1, 1], xlabel = L"x", ylabel = L"f(x)", backgroundcolor = :transparent)
scatter!(ax, xdata, ydata; color=:black, label="all data")
scatter!(ax, xtraining, ytraining; color=:red, label="training data")
axislegend(ax, location=:rt)
save("../figures/06p03NeuralNetworkWithJuliaPart204.png", fig)
fig
```



Choosing a model

```
In [51]: model_training = Chain(Dense(1 => 5,tanh),
    Dense(5 => 5,tanh),
    Dense(5=>1))

# Initialize the model weights and state
rng = Random.default_rng()
ps, ls = Lux.setup(rng, model_training)

((layer_1 = (weight = Float32[2.8126822; 0.43456358; ... ; -2.8000252; 0.013
202454;;], bias = Float32[-0.15028763, 0.7801498, 0.74896264, 0.18760526,
-0.06775844]), layer_2 = (weight = Float32[-0.83308905 0.96173835 ... -0.442
1943 0.09656086; 1.0911583 -0.016539475 ... -0.048167467 1.20371; ... ; -0.960
8359 -0.8736775 ... 0.011510532 -1.0125556; -1.2570175 0.5195003 ... -1.077008
5 -1.0715384], bias = Float32[0.3824637, -0.33159572, -0.4125466, -0.28877
118, 0.37937763]), layer_3 = (weight = Float32[0.0795809 0.2588633 ... -0.13
799204 -0.3825596], bias = Float32[0.36268836])), (layer_1 = NamedTuple(),
layer_2 = NamedTuple(), layer_3 = NamedTuple()))
```

Training the model

```
In [52]: # The forward function
function loss_training(ps, ytraining)
    ypred,_=model_training(xtraining', ps, ls)
    loss = 0.5 * mean((ypred.-ytraining').^2)
    return loss
end

callback = function (p, l)
    if length(loss_history) % 100 == 0
        println("Iteration: $(p.iter), Loss: $l")
    end
end
```

```
        push!(loss_history, l)
        return false
    end

    adtype = Optimization.AutoZygote()
    optf = Optimization.OptimizationFunction(loss_training, adtype)
    optprob = Optimization.OptimizationProblem(optf, ComponentArray(ps), ytrain)

    # Initialize a vector to store loss values
    loss_history = Float64[]
    neural_network = Optimization.solve(optprob, OptimizationOptimisers.Adam(),
    ps=neural_network.u
```

Iteration: 1, Loss: 0.254238502440033
Iteration: 101, Loss: 0.07229499495070398
Iteration: 201, Loss: 0.03990792317391949
Iteration: 301, Loss: 0.010942156530960933
Iteration: 401, Loss: 0.006577881702759839
Iteration: 501, Loss: 0.005331070456983286
Iteration: 601, Loss: 0.0048544594725865664
Iteration: 701, Loss: 0.004045458991332544
Iteration: 801, Loss: 0.001665626523497666
Iteration: 901, Loss: 0.0015013561457149415
Iteration: 1001, Loss: 0.0014659504005923151
Iteration: 1101, Loss: 0.0014352300758489894
Iteration: 1201, Loss: 0.0014097859072605065
Iteration: 1301, Loss: 0.0013800216195493532
Iteration: 1401, Loss: 0.0014331328102896485
Iteration: 1501, Loss: 0.001334330560734152
Iteration: 1601, Loss: 0.0013342386397238331
Iteration: 1701, Loss: 0.0012965366734857535
Iteration: 1801, Loss: 0.001280829715053274
Iteration: 1901, Loss: 0.0012653165847344937
Iteration: 2001, Loss: 0.0012527994110926378
Iteration: 2101, Loss: 0.0012395955789379878
Iteration: 2201, Loss: 0.0012289550461712286
Iteration: 2301, Loss: 0.0012179233403037018
Iteration: 2401, Loss: 0.0012074767421484834
Iteration: 2501, Loss: 0.0011992826965172448
Iteration: 2601, Loss: 0.0011902359073348557
Iteration: 2701, Loss: 0.0012151775066386377
Iteration: 2801, Loss: 0.0011753500848959116
Iteration: 2901, Loss: 0.0017454294946601767
Iteration: 3001, Loss: 0.001162728782419219
Iteration: 3101, Loss: 0.0011562587122034046
Iteration: 3201, Loss: 0.001161909419787027
Iteration: 3301, Loss: 0.0011457982653725975
Iteration: 3401, Loss: 0.0011402309434027256
Iteration: 3501, Loss: 0.0011367862216208418
Iteration: 3601, Loss: 0.001131071560358759
Iteration: 3701, Loss: 0.001139243668801222
Iteration: 3801, Loss: 0.0011227910019801196
Iteration: 3901, Loss: 0.0011182927425046874
Iteration: 4001, Loss: 0.0011152033950429267
Iteration: 4101, Loss: 0.0011108569965365192
Iteration: 4201, Loss: 0.0011109210981174987
Iteration: 4301, Loss: 0.001104198877661217
Iteration: 4401, Loss: 0.0011003781437609896
Iteration: 4501, Loss: 0.001099435273768795
Iteration: 4601, Loss: 0.0010947896642355714
Iteration: 4701, Loss: 0.0010913239851429188
Iteration: 4801, Loss: 0.0011537611098158648
Iteration: 4901, Loss: 0.0010859392236620355
Iteration: 5001, Loss: 0.0010826335520132716
Iteration: 5101, Loss: 0.0010911140406750677
Iteration: 5201, Loss: 0.0010770149312058896
Iteration: 5301, Loss: 0.0010735359104391746
Iteration: 5401, Loss: 0.0010710468643414072
Iteration: 5501, Loss: 0.0010675755037977764
Iteration: 5601, Loss: 0.0010721074364298203
Iteration: 5701, Loss: 0.0010621841906657333
Iteration: 5801, Loss: 0.001059610709794548
Iteration: 5901, Loss: 0.0010572111258315673

```

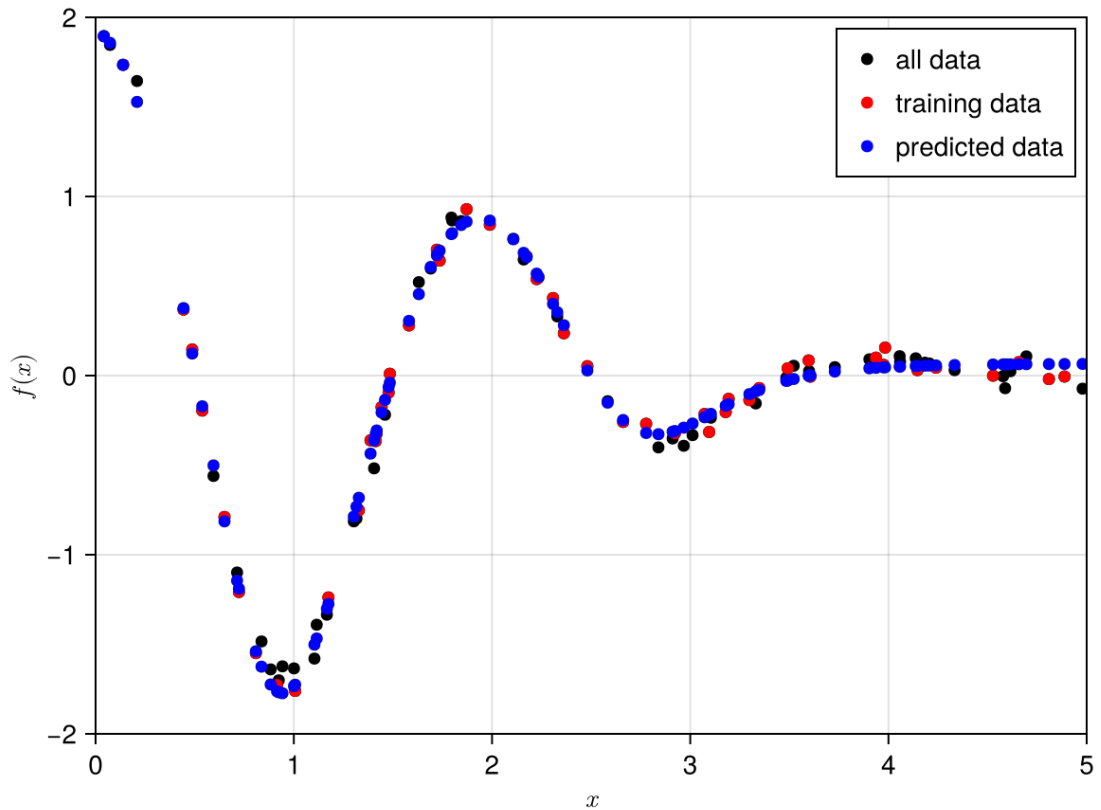
Iteration: 6001, Loss: 0.0010539161458609412
Iteration: 6101, Loss: 0.0011187438240125682
Iteration: 6201, Loss: 0.0010485956613090876
Iteration: 6301, Loss: 0.0010453752262837004
Iteration: 6401, Loss: 0.0010433351874746
Iteration: 6501, Loss: 0.0010400826437281325
Iteration: 6601, Loss: 0.00104016745401701
Iteration: 6701, Loss: 0.0010347343355620827
Iteration: 6801, Loss: 0.0010314265966155573
Iteration: 6901, Loss: 0.0010302897426442616
Iteration: 7001, Loss: 0.0010263634506221583
Iteration: 7101, Loss: 0.0010230687056580368
Iteration: 7201, Loss: 0.0010220498538333614
Iteration: 7301, Loss: 0.0010174433626986622
Iteration: 7401, Loss: 0.0010155578775958446
Iteration: 7501, Loss: 0.0010120589023774871
Iteration: 7601, Loss: 0.0010089211931732772
Iteration: 7701, Loss: 0.0010074768812870652
Iteration: 7801, Loss: 0.0010041672160654319
Iteration: 7901, Loss: 0.0010013304128923272
Iteration: 8001, Loss: 0.000999614277637032
Iteration: 8101, Loss: 0.0009969177566645664
Iteration: 8201, Loss: 0.0014175607508784922
Iteration: 8301, Loss: 0.000992951423664594
Iteration: 8401, Loss: 0.0009906715129632925
Iteration: 8501, Loss: 0.0010205658693665162
Iteration: 8601, Loss: 0.0009872016727997964
Iteration: 8701, Loss: 0.0009852054028214158
Iteration: 8801, Loss: 0.0009857200075072514
Iteration: 8901, Loss: 0.0009823703327930186
Iteration: 9001, Loss: 0.0009806122085842539
Iteration: 9101, Loss: 0.0009828104644304591
Iteration: 9201, Loss: 0.000978004398366711
Iteration: 9301, Loss: 0.0009764121894658149
Iteration: 9401, Loss: 0.0009759421307201222
Iteration: 9501, Loss: 0.0009740323185252861
Iteration: 9601, Loss: 0.0010068907617966575
Iteration: 9701, Loss: 0.0009719237199516832
Iteration: 9801, Loss: 0.0009704765923176664
Iteration: 9901, Loss: 0.0009691584822377256
Iteration: 10000, Loss: 0.0009691566280979896
ComponentVector{Float32}(layer_1 = (weight = Float32[0.9475966; 0.3626202
6; ... ; -2.8164363; 1.345231;], bias = Float32[-3.2317116, 0.7757226, 2.04
9569, 1.275323, -1.0862648]), layer_2 = (weight = Float32[-4.070168 -0.288
90327 ... 1.5390081 -0.122155175; 1.7381042 0.10027236 ... 0.016227687 2.87016
13; ... ; -1.5142974 -0.81499857 ... -0.8636703 -4.3408318; -1.5063072 -0.4752
1445 ... -2.0323236 -2.5473473], bias = Float32[-1.0695869, -0.36234817, 0.4
8897162, -0.2130592, -0.6517164]), layer_3 = (weight = Float32[3.3093107
1.791892 ... -0.4240285 -1.6611755], bias = Float32[0.43222436]))

```

```

In [53]: fig = Figure(backgroundcolor = :transparent)
ax = CairoMakie.Axis(fig[1, 1], xlabel = L"x", ylabel = L"f(x)", background
scatter!(ax, xdata, ydata; color=:black, label="all data")
scatter!(ax, xtraining, ytraining; color=:red, label="training data")
scatter!(ax, xdata, model_training(xdata', ps, ls)[1][:]; color=:blue, label="p
axislegend(ax, location=:rt)
save("../figures/06p03NeuralNetworkWithJuliaPart205.png", fig)
fig

```

As expected, if you randomly pick your training data for $0 < x < 5$ and you train your model in this region, then your prediction in this region is going to be good.

You can find the mean of the square of the error between the model and the validation dataset by using the code below

```
In [54]: validation_indices=setdiff(1:N_SAMPLES,training_indices)
ypred,_=model_training(xdata[validation_indices]', ps, ls)
mean((ypred.-ydata[validation_indices]').^2)
```

0.004846285932211208

What we have done so far is to train the model with data in the region $x \in [0, 5]$ and then predict the data $x \in [0, 5]$. This is a bit pointless because if we already have the data within a certain parameter space, there is no real reason to predict within the same region. Ideally we would like to use our model to predict data outside the training region. Now we will try to train the model with only data for $x < x_0$ and then use it predict for $x > x_0$. What we are trying to do is to see if the model can predict beyond the training region, i.e. when we use it to extrapolate our data. Let's see what happens. We will pick $x_0 = 2$.

```
In [55]: training_region=2.0
training_indices = findall(xdata .< training_region)
xtraining=xdata[training_indices]
ytraining=ydata[training_indices]
```

45-element Vector{Float64}:

```

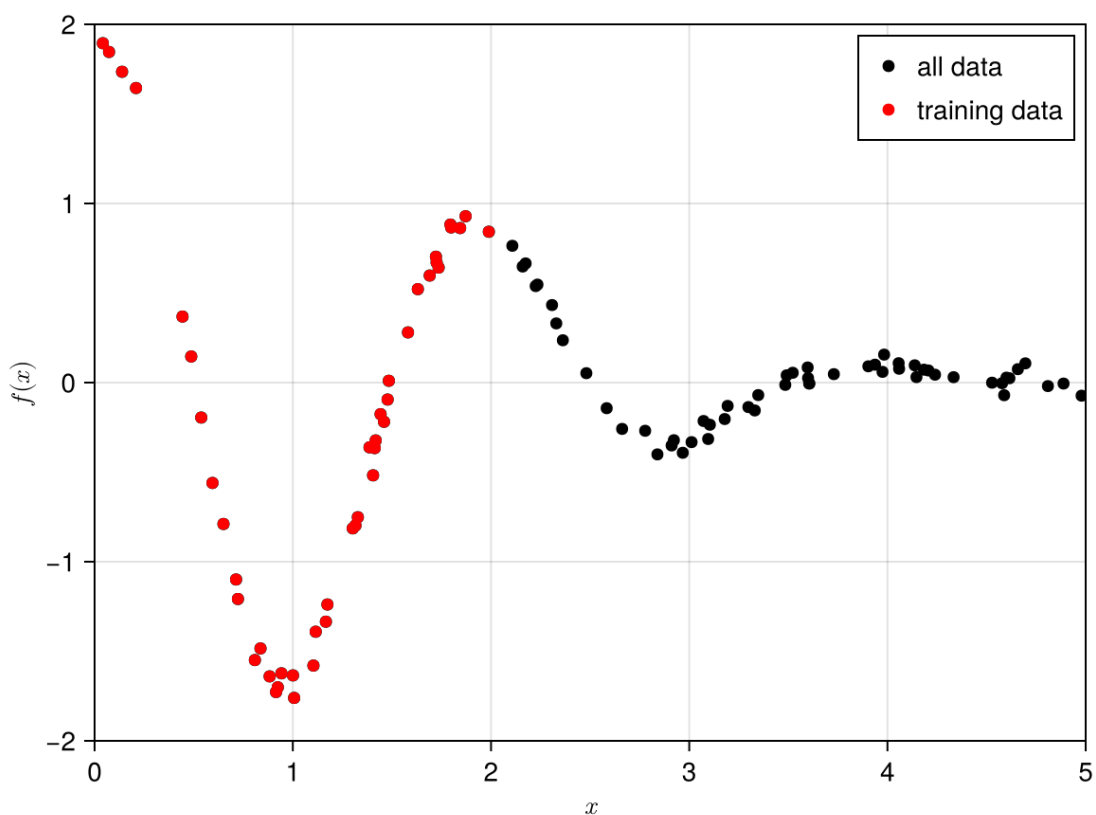
-0.17619228051694247
-1.7274756929111315
-0.7978215087224122
-0.19509082769413835
-1.7603478855316192
 0.8662223312958148
 0.5976289516342747
-0.8136904227827861
 0.36848817771805303
 0.670783980256972
  ⋮
 1.6445148002465255
-0.36566488195999997
-1.238794325611893
 0.14568669520657762
-1.0992153985638868
 0.9286929364896507
 0.8821441593661695
 1.7352061081875674
-0.7892209947566635

```

```

In [56]: fig = Figure(backgroundcolor = :transparent)
ax = CairoMakie.Axis(fig[1, 1], xlabel = L"x", ylabel = L"f(x)", background
scatter!(ax, xdata, ydata; color=:black, label="all data")
scatter!(ax, xtraining, ytraining; color=:red, label="training data")
axislegend(ax, location=:rt)
save("../figures/06p03NeuralNetworkWithJuliaPart202.png", fig)
fig

```



```

In [57]: model_training = Chain(Dense(1 => 5, tanh),
Dense(5 => 5, tanh),
Dense(5 => 1))

```

```
# Initialize the model weights and state
```

```
rng = Random.default_rng()
ps, ls = Lux.setup(rng, model_training)
```

```
((layer_1 = (weight = Float32[2.070374; 0.8509132; ... ; 2.0901117; 1.639633
3;;], bias = Float32[-0.40130138, -0.9151753, 0.53922653, 0.4958433, -0.48
764384]), layer_2 = (weight = Float32[-0.8765004 -0.577796 ... -0.53373903 -
1.0062816; 1.1911608 -0.37699553 ... -1.0417581 -0.7501479; ... ; 0.47040224 -
0.20209293 ... 1.2774144 0.3785936; 1.271577 1.1008525 ... -0.3085536 -0.16852
304], bias = Float32[0.30622965, -0.4079655, 0.330182, -0.22390641, 0.0919
3247]), layer_3 = (weight = Float32[0.1711138 -0.5143709 ... 0.24493201 -0.6
530002], bias = Float32[-0.24454579])), (layer_1 = NamedTuple(), layer_2 =
NamedTuple(), layer_3 = NamedTuple()))
```

In [58]:

```
# The loss function
```

```
function loss_training(ps, ytraining)
    ypred, _ = model_training(xtraining', ps, ls)
    loss = 0.5 * mean((ypred.-ytraining').^2)
    return loss
end
```

```
callback = function (p, l)
    if length(loss_history) % 100 == 0
        println("Iteration: $(p.iter), Loss: $l")
    end
    push!(loss_history, l)
    return false
end
```

```
adtype = Optimization.AutoZygote()
optf = Optimization.OptimizationFunction(loss_training, adtype)
optprob = Optimization.OptimizationProblem(optf, ComponentArray(ps), ytrai

# Initialize a vector to store loss values
loss_history = Float64[]
neural_network = Optimization.solve(optprob, OptimizationOptimisers.Adam(
ps=neural_network.u
```

Iteration: 1, Loss: 0.4686471614065544
Iteration: 101, Loss: 0.002950726356271961
Iteration: 201, Loss: 0.0021873024346733745
Iteration: 301, Loss: 0.0020889237483738957
Iteration: 401, Loss: 0.002041613366217326
Iteration: 501, Loss: 0.001971482870789316
Iteration: 601, Loss: 0.0017586463545223592
Iteration: 701, Loss: 0.001572416314150228
Iteration: 801, Loss: 0.001450814645188383
Iteration: 901, Loss: 0.001384659038662949
Iteration: 1001, Loss: 0.0013532741222446808
Iteration: 1101, Loss: 0.0013369435153659374
Iteration: 1201, Loss: 0.0013271221888486874
Iteration: 1301, Loss: 0.0013204042111605594
Iteration: 1401, Loss: 0.0013150131632756895
Iteration: 1501, Loss: 0.0013103122923409304
Iteration: 1601, Loss: 0.0013059792246536894
Iteration: 1701, Loss: 0.001302069750057689
Iteration: 1801, Loss: 0.0012980756919141535
Iteration: 1901, Loss: 0.0012951050901483384
Iteration: 2001, Loss: 0.0012910378347435131
Iteration: 2101, Loss: 0.001294725620131405
Iteration: 2201, Loss: 0.0012847671899967256
Iteration: 2301, Loss: 0.001281880360770358
Iteration: 2401, Loss: 0.0012792509771959888
Iteration: 2501, Loss: 0.0012767053543393493
Iteration: 2601, Loss: 0.0012809930911369134
Iteration: 2701, Loss: 0.0012721308989034428
Iteration: 2801, Loss: 0.0012700197319954667
Iteration: 2901, Loss: 0.001269979669346181
Iteration: 3001, Loss: 0.0012662652520240459
Iteration: 3101, Loss: 0.0012976673543733128
Iteration: 3201, Loss: 0.0012630201529288702
Iteration: 3301, Loss: 0.001261535134746198
Iteration: 3401, Loss: 0.0012601427287938871
Iteration: 3501, Loss: 0.0012590182513924038
Iteration: 3601, Loss: 0.0012576636957638682
Iteration: 3701, Loss: 0.0012565226017218977
Iteration: 3801, Loss: 0.0012554916196188999
Iteration: 3901, Loss: 0.0012544862080675011
Iteration: 4001, Loss: 0.0012577534824522593
Iteration: 4101, Loss: 0.0012526832379034067
Iteration: 4201, Loss: 0.0012518619747278633
Iteration: 4301, Loss: 0.0012513989709884905
Iteration: 4401, Loss: 0.0012503754014296297
Iteration: 4501, Loss: 0.0014687658638972876
Iteration: 4601, Loss: 0.001249069369536853
Iteration: 4701, Loss: 0.0012484692670593454
Iteration: 4801, Loss: 0.0012500741715152015
Iteration: 4901, Loss: 0.0012473900156854574
Iteration: 5001, Loss: 0.0012468945622742006
Iteration: 5101, Loss: 0.00124673811823784
Iteration: 5201, Loss: 0.0012460095158245517
Iteration: 5301, Loss: 0.001245599555034259
Iteration: 5401, Loss: 0.0012454781652249053
Iteration: 5501, Loss: 0.001244871031589429
Iteration: 5601, Loss: 0.0016782767116799296
Iteration: 5701, Loss: 0.0012442380578015085
Iteration: 5801, Loss: 0.0012439343203301788
Iteration: 5901, Loss: 0.001296155186892974

```

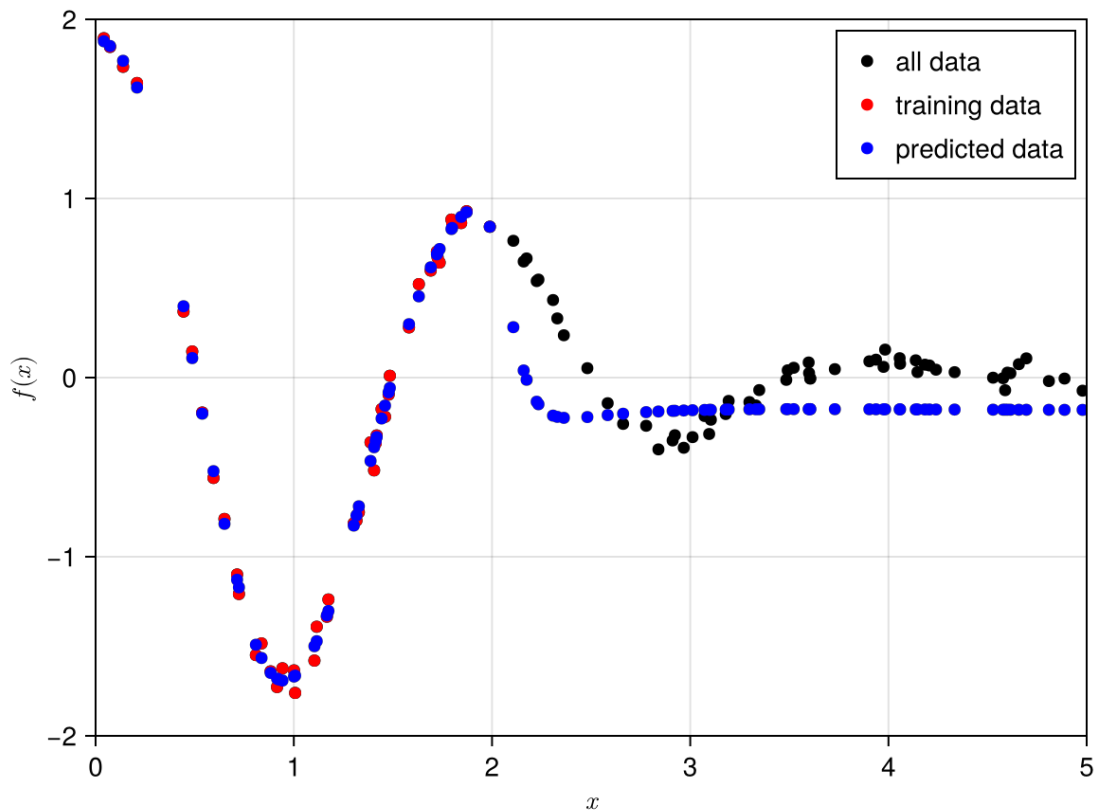
Iteration: 6001, Loss: 0.0012434288025999012
Iteration: 6101, Loss: 0.0012431742716201503
Iteration: 6201, Loss: 0.001254411297631859
Iteration: 6301, Loss: 0.0012427987829528625
Iteration: 6401, Loss: 0.0012425491058224523
Iteration: 6501, Loss: 0.0012423501820739702
Iteration: 6601, Loss: 0.0012422241098186643
Iteration: 6701, Loss: 0.0012420286976348425
Iteration: 6801, Loss: 0.0012420200039759497
Iteration: 6901, Loss: 0.0012417798996613761
Iteration: 7001, Loss: 0.0012415829347575437
Iteration: 7101, Loss: 0.0012638812397431913
Iteration: 7201, Loss: 0.001241345706963377
Iteration: 7301, Loss: 0.0012412060646217527
Iteration: 7401, Loss: 0.0012412405536985874
Iteration: 7501, Loss: 0.0012409953962480421
Iteration: 7601, Loss: 0.0012442153263227842
Iteration: 7701, Loss: 0.0012408149119094328
Iteration: 7801, Loss: 0.0012406974540122777
Iteration: 7901, Loss: 0.001240764266470694
Iteration: 8001, Loss: 0.0012405380252435586
Iteration: 8101, Loss: 0.0013742594839622868
Iteration: 8201, Loss: 0.0012404097850049994
Iteration: 8301, Loss: 0.001240296916076895
Iteration: 8401, Loss: 0.0012482319589903466
Iteration: 8501, Loss: 0.0012401832251994788
Iteration: 8601, Loss: 0.0012400777504675902
Iteration: 8701, Loss: 0.0012403303762421583
Iteration: 8801, Loss: 0.0012399756453978143
Iteration: 8901, Loss: 0.0012398762835188369
Iteration: 9001, Loss: 0.0012400835902592127
Iteration: 9101, Loss: 0.001239776729041699
Iteration: 9201, Loss: 0.0012415499638772946
Iteration: 9301, Loss: 0.0012397079871003997
Iteration: 9401, Loss: 0.0012395904540699679
Iteration: 9501, Loss: 0.0012914928383484829
Iteration: 9601, Loss: 0.001239527164311306
Iteration: 9701, Loss: 0.0012394191436490161
Iteration: 9801, Loss: 0.0014051531030990225
Iteration: 9901, Loss: 0.001239330837270826
Iteration: 10000, Loss: 0.0012392342607370067
ComponentVector{Float32}(layer_1 = (weight = Float32[2.665926; 1.1251196;
... ; 1.3253365; 1.8408302;;], bias = Float32[-0.95943457, -1.5681283, 1.179
8577, 1.383915, -0.7259732]), layer_2 = (weight = Float32[-1.3812964 -1.37
5309 ... 0.0053010136 -1.5502324; 1.6402862 -1.2077146 ... -0.9779316 -0.42473
12; ... ; 1.1548569 -5.41425 ... 2.4765456 0.98229724; 1.562189 0.8214901 ... 0.
10796843 0.11923442], bias = Float32[0.85215396, -0.4018001, 0.49734956,
0.85349375, 0.50053585]), layer_3 = (weight = Float32[0.7923084 -0.9992167
4 ... 0.76581377 -0.8059152], bias = Float32[-0.062539026]))

```

```

In [59]: fig = Figure(backgroundcolor = :transparent)
ax = CairoMakie.Axis(fig[1, 1], xlabel = L"x", ylabel = L"f(x)", background
scatter!(ax, xdata, ydata; color=:black, label="all data")
scatter!(ax, xtraining, ytraining; color=:red, label="training data")
scatter!(ax, xdata, model_training(xdata', ps, ls)[1][:]; color=:blue, label="p
axislegend(ax, location=:rt)
#save("../figures/06p03NeuralNetworkWithJuliaPart202.png", fig)
fig

```



Finding the mean squared error between training and validation dataset.

```
In [60]: validation_indices=setdiff(1:N_SAMPLES,training_indices)
ypred,_=model_training(xdata[validation_indices]', ps, ls)
mean((ypred.-ydata[validation_indices]').^2)
```

0.08594844837399783

Class Demo 01: Construct a Neural Network using Lux in Julia to fit a limited section of the data in "Data06.csv". Then try using this Neural Network to predict the data beyond the training region

In [61]: